

CHAPTER FIVE: SYSTEM DESIGN

5.1 Introduction

This chapter details the technical design of Study Cue, transforming the requirements into a functional architecture. It covers the system architecture, process modeling, database design, and user interface layouts.

5.2 Architecture of the Proposed System

Study Cue follows a modern **Full-Stack Decoupled Architecture**, splitting concerns between a reactive frontend and a logic-heavy backend.

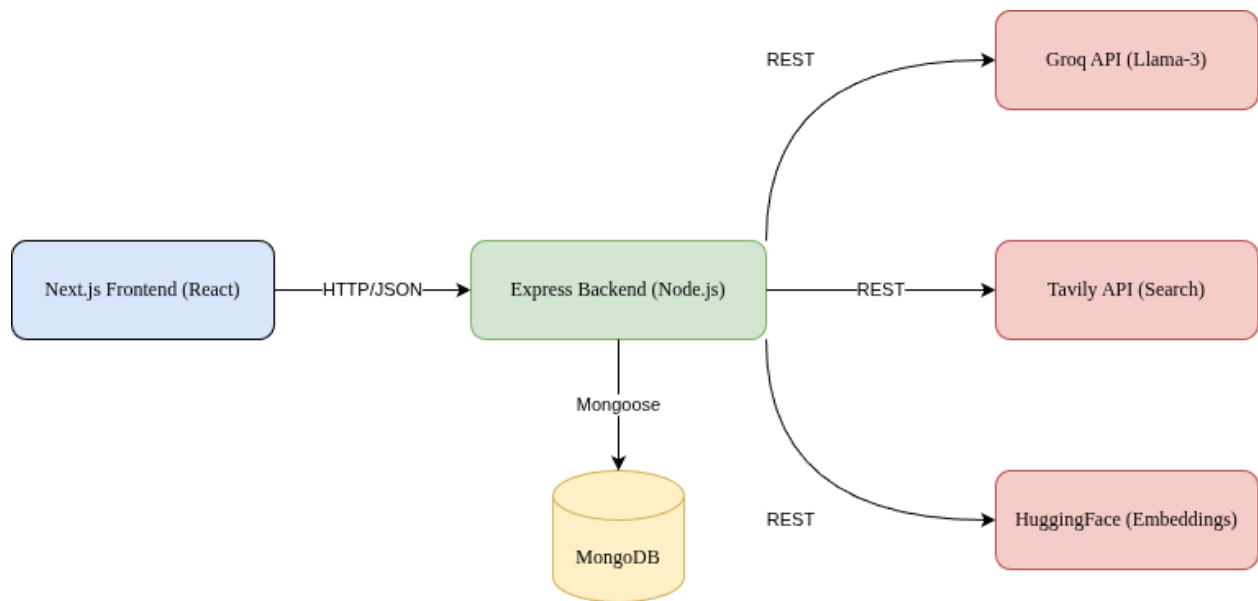


Figure 5.1: Proposed System Architecture Diagram showing the decoupled full-stack infrastructure and integration with AI APIs.

The system components are as follows:

- **Frontend (Next.js 14):** A responsive web application using React Server Components for performance and Shadcn UI for a premium aesthetic.
- **Backend (Node.js/Express):** A RESTful API that orchestrates logic between the database and external AI services.
- **Database (MongoDB):** A NoSQL database for flexible storage of study plans, chat histories, and notes.

- **AI Services:**
 - **Groq:** LLM inference (Llama-3) for text generation and reasoning.
 - **HuggingFace:** Sentence embeddings for PDF search.
 - **Tavily:** Academic-focused web search.

5.3 System Process Modeling

The following sequence diagrams illustrate the core workflows of the Study Cue platform, specifically the AI-driven study planning and the Retrieval-Augmented Generation (RAG) process for documents curation and analysis.

5.3.1 User Registration and Authentication Process

The authentication system leverages NextAuth.js to provide secure access via OAuth (Google) and Credentials-based protocols, ensuring user data privacy and session persistence.

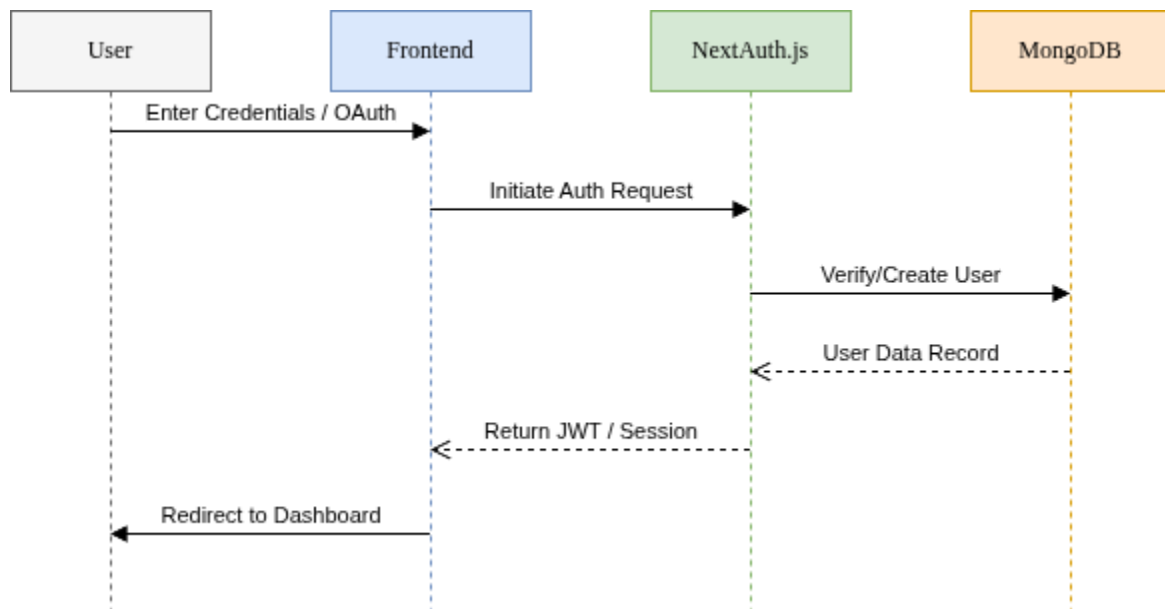


Figure 5.2: Sequence Diagram illustrating the secure user registration and authentication flow.

5.3.2 Study Plan Generation Process

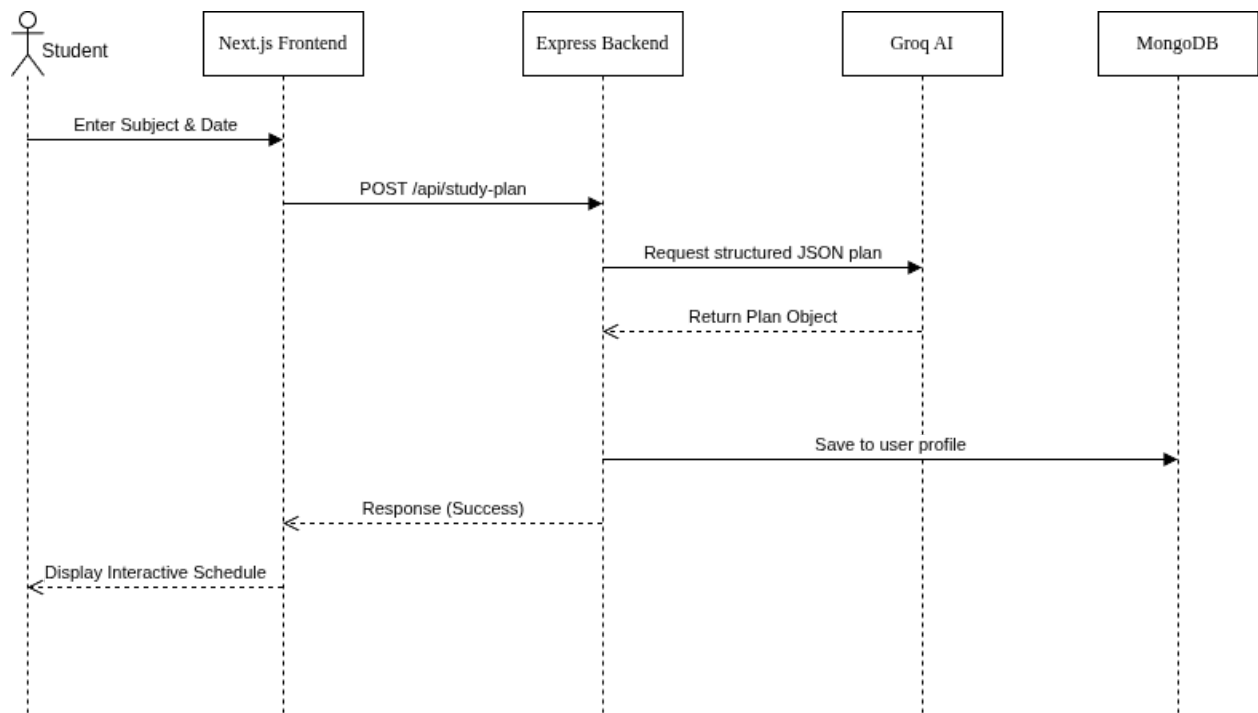


Figure 5.3: Sequence Diagram illustrating the AI-driven study plan generation workflow.

5.3.3 PDF Chat (RAG) Process

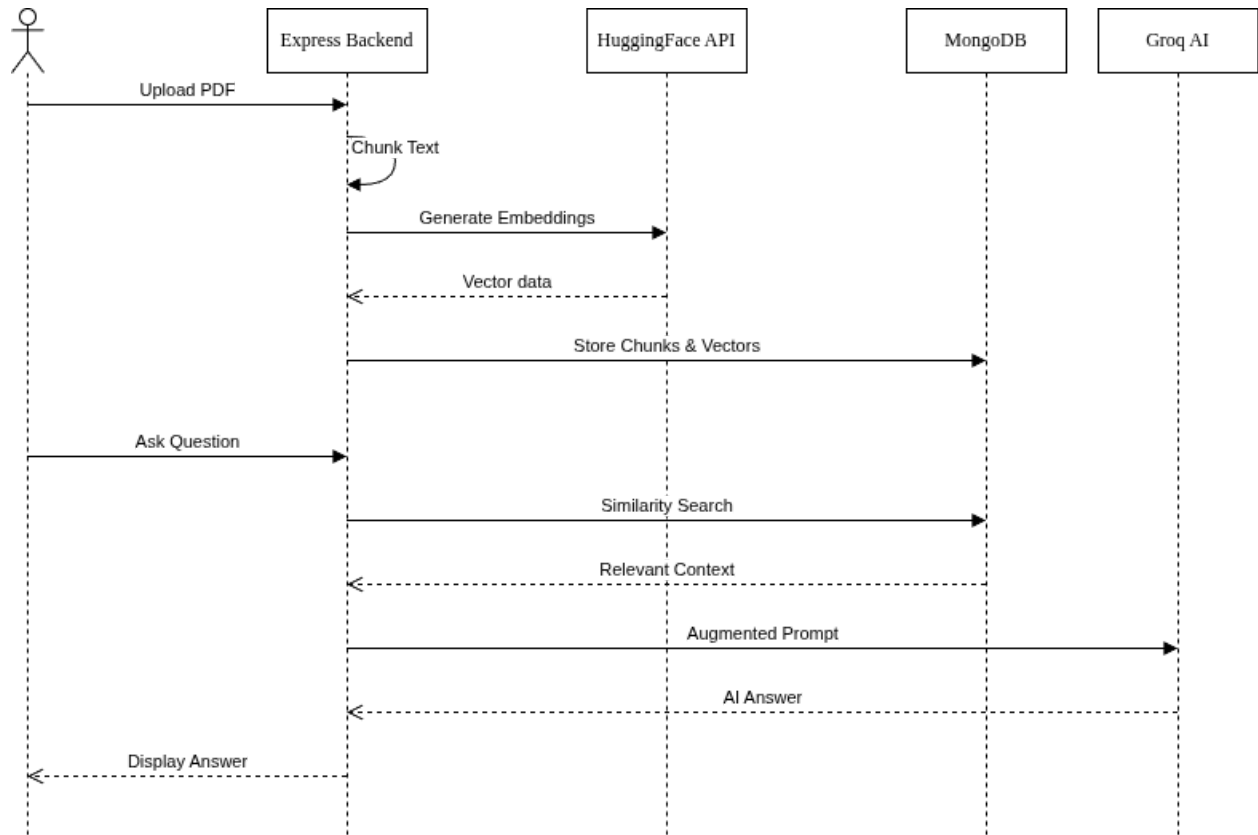


Figure 5.4: Sequence Diagram illustrating the Retrieval-Augmented Generation (RAG) process for interactive PDF chat.

5.3.4 Resource Search and Curation Process

Study Cue automates the discovery of academic materials by integrating real-time web search with AI summarization.

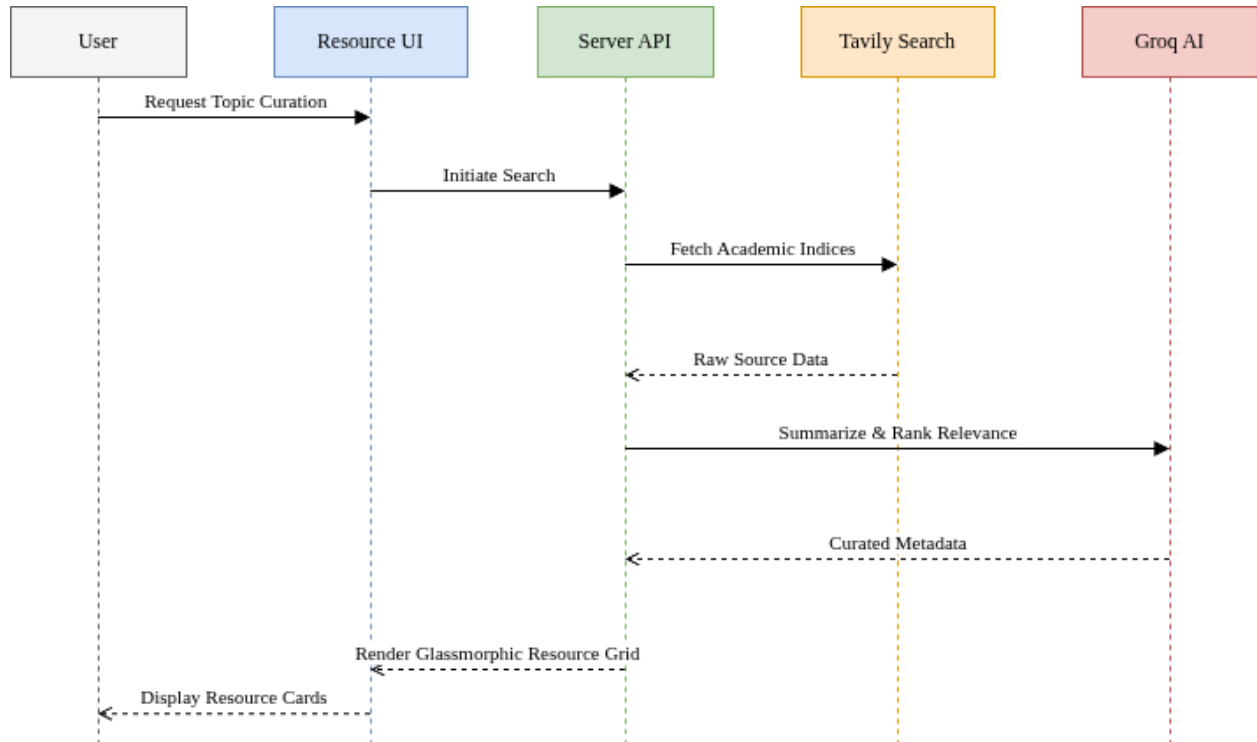


Figure 5.5: Sequence Diagram illustrating the automated resource discovery and curation workflow.

5.3.5 Notes Creation and Management Process

The platform allows users to capture insights from AI chats or manual study sessions into a centralized "Neural Library" where the user can edit or modify the notes.

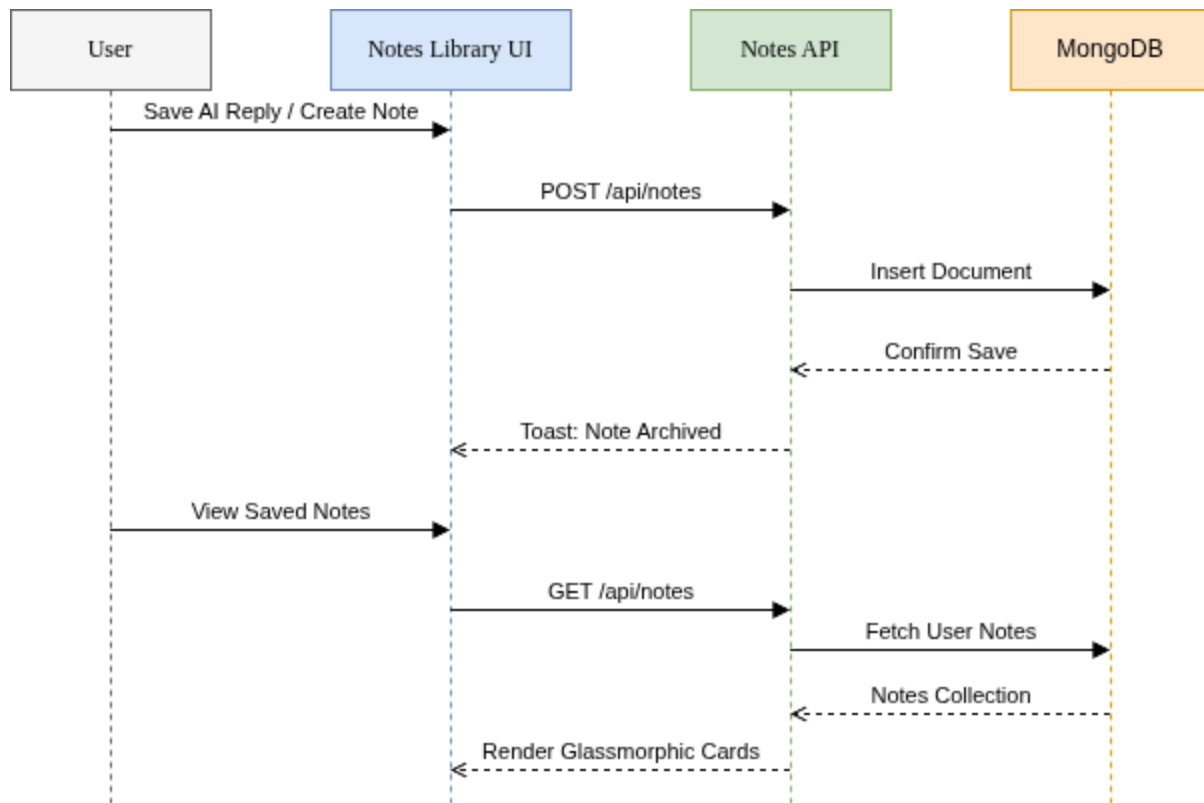


Figure 5.6: Sequence Diagram illustrating the note creation, archival, and retrieval process.

5.3.6 Community Q&A Flow

The Community Q&A module allows authenticated students to publish questions and resource folders to a shared academic feed.

Process:

1. An authenticated student submits a question (title, body, tags) via the Community page.
2. The Next.js frontend sends a POST request to the Express backend (</api/community/questions>).
3. The backend validates the payload, attaches the author's ID and name from the JWT session, and saves a new [CommunityQuestion](#) document to MongoDB.
4. The question is immediately visible in the community feed, sorted by recency or upvote count.

- Other students can post replies (threaded, with optional **replyTo** nesting) or upvote the question.
- The platform optionally generates an AI reply via the Groq API to provide an immediate starting answer.
- Admin users can soft-delete or review reported posts via the Admin Panel.

5.4 Database Design

The database uses MongoDB collections to handle relational and semi-structured data.

5.4.1 Data Dictionary (Core Collections)

Collection: **Users**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique user identifier
name	String	Required	Full name of the user
email	String	Required, Unique	Login identifier
password	String	Required	Hashed credentials
subjects	Array	Optional	List of academic interests
savedPlans	Array	Embedded	List of generated study plans
savedResources	Array	Embedded	Curated materials saved by user
stats	ObjectId	Foreign Key	Reference to StudyStats

Collection: **StudyPlans**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique plan identifier

Field	Data Type	Constraint	Description
userId	ObjectId	Foreign Key	Owner of the plan
overview	Object	Required	Subject, duration, and exam date
weeklyPlans	Array	Required	Temporal schedule with tasks
isActive	Boolean	Default: True	Current monitoring status
progress	Number	0-100	Completion percentage
lastUpdated	Date	Default: Now	Last activity timestamp

Collection: **Notes**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique note identifier
userId	ObjectId	Foreign Key	Author of the note
title	String	Required	Display title
content	Array	Blocks	JSON-based block structure
path	String	Default: '/'	Hierarchy placement
isArchived	Boolean	Default: False	For soft deletion

Collection: **StudyStats**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique stats identifier
userId	ObjectId	Foreign Key	User performance link
totalHours	Number	Accumulated	Total time in Focus mode

Field	Data Type	Constraint	Description
currentStreak	Number	Daily	Consecutive study days
dailySessions	Map	Complex	Log of Pomodoro completions

Collection: **PdfDocuments**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique document identifier
userId	ObjectId	Foreign Key	Owner of the document
title	String	Required	Filename for display
chunks	Array	Required	Text segments for RAG context
embeddings	Array	Required	Vector data for semantic search
chatHistory	Array	Optional	List of previous Q&A pairs

Collection: **CuratedResources**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique resource set identifier
userId	String	Required	User who requested search
topic	String	Required	The search keyword/topic
resources	Array	Required	URLs, types, and AI summaries
lastUpdated	Date	Default: Now	Freshness timestamp

Collection: **CommunityQuestions**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique question identifier
title	String	Required	Short display title of the question
body	String	Required	Full question content (supports Markdown)
authorId	String	Required	ID of the posting user
authorName	String	Required	Display name of the author
tags	Array	Optional	Topic tags (e.g., "study-tip", "physics")
upvotes	Number	Default: 0	Net upvote count
upvotedBy	Array	Optional	IDs of users who upvoted
replies	Array	Embedded	Threaded replies from peers or AI
isDeleted	Boolean	Default: False	Soft delete flag for admin moderation
isReported	Boolean	Default: False	Flagged for admin review
createdAt	Date	Default: Now	Submission timestamp

Collection: **CommunityResources**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique resource folder identifier
authorId	String	Required	ID of the publishing user

Field	Data Type	Constraint	Description
authorName	String	Required	Display name of the publisher
topic	String	Required	Folder title / subject area
description	String	Optional	Brief folder summary
resourceType	String	Enum:	Structural type for nesting
parentId	String	Optional	Parent folder ID for subfolder nesting
tags	Array	Optional	Descriptive tags
resources	Array	Required	List of curated links with title, URL, type, description
upvotes	Number	Default: 0	Community upvote count
saves	Number	Default: 0	Number of times cloned to personal libraries
savedBy	Array	Optional	IDs of users who saved this folder
isDeleted	Boolean	Default: False	Soft delete for moderation
createdAt	Date	Default: Now	Publication timestamp

Collection: **FocusSessions**

Field	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique session identifier
userId	String	Required	ID of the student running the session
userName	String	Required	Display name of the student

Field	Data Type	Constraint	Description
subject	String	Default:	Subject being studied in this session
durationMinutes	Number	Required	Intended Pomodoro duration (e.g., 25 mins)
startTime	Date	Default: Now	Session start timestamp
completedMinutes	Number	Default: 0	Actual minutes of focused study completed
status	String	Active/Completed/ Abandoned	Current state of the session

5.5 User Interface (UI) Design

The UI is designed to be "Calm and Focused," leveraging a glassmorphic aesthetic to minimize cognitive load while maintaining a premium, high-tech feel. All interfaces prioritize dark modes with vibrant accents to guide user attention.

5.5.1 Dashboard Interface

The central hub for student activity, the Dashboard integrates temporal tracking with real-time, focus session and progress monitoring.

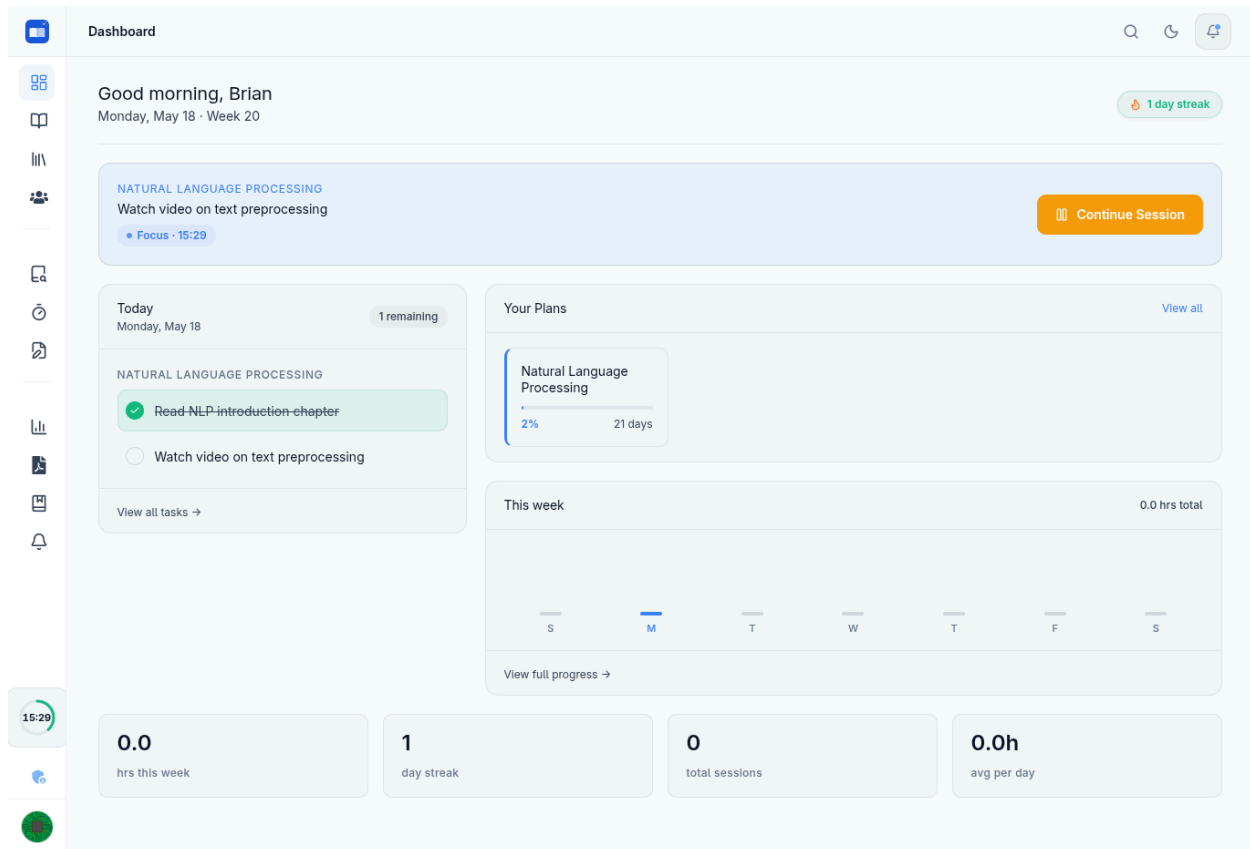


Figure 5.8: The Study Cue Dashboard providing a summary of active courses, progress tracking via the "Neural Roadmap" chart, and an integrated Pomodoro timer for focused study sessions.

5.5.2 Study Library (Resources and PDF Management)

The Study Library is the organizational hub where users manage folders, projects, and AI-curated resource collections. It supports PDF uploads (linked to the PDF Workshop for AI-powered chat), manual resource management, community publishing, and serves as the shared folder structure used by Study Plans and the Research Hub.

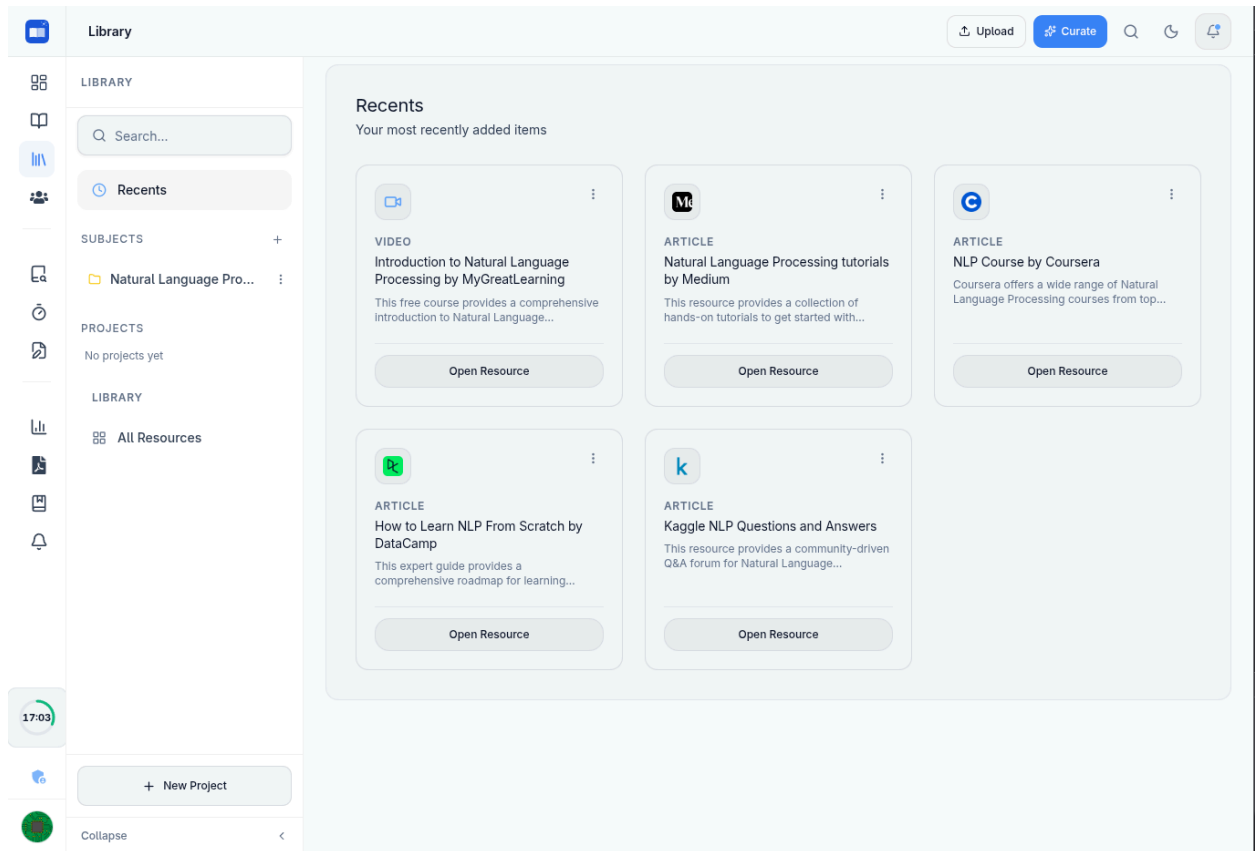


Figure 5.9: The PDF Workshop featuring a grid design for resource management and AI-assisted analysis. The interface includes an AI Study Assistant with the ability to archive insights directly to notes.

5.5.3 Research Hub (Resource Library)

The Research Hub lets users save resources by URL (with auto-extracted metadata) or enter a topic to AI-curate a collection of free learning resources using web search and LLM-based selection.

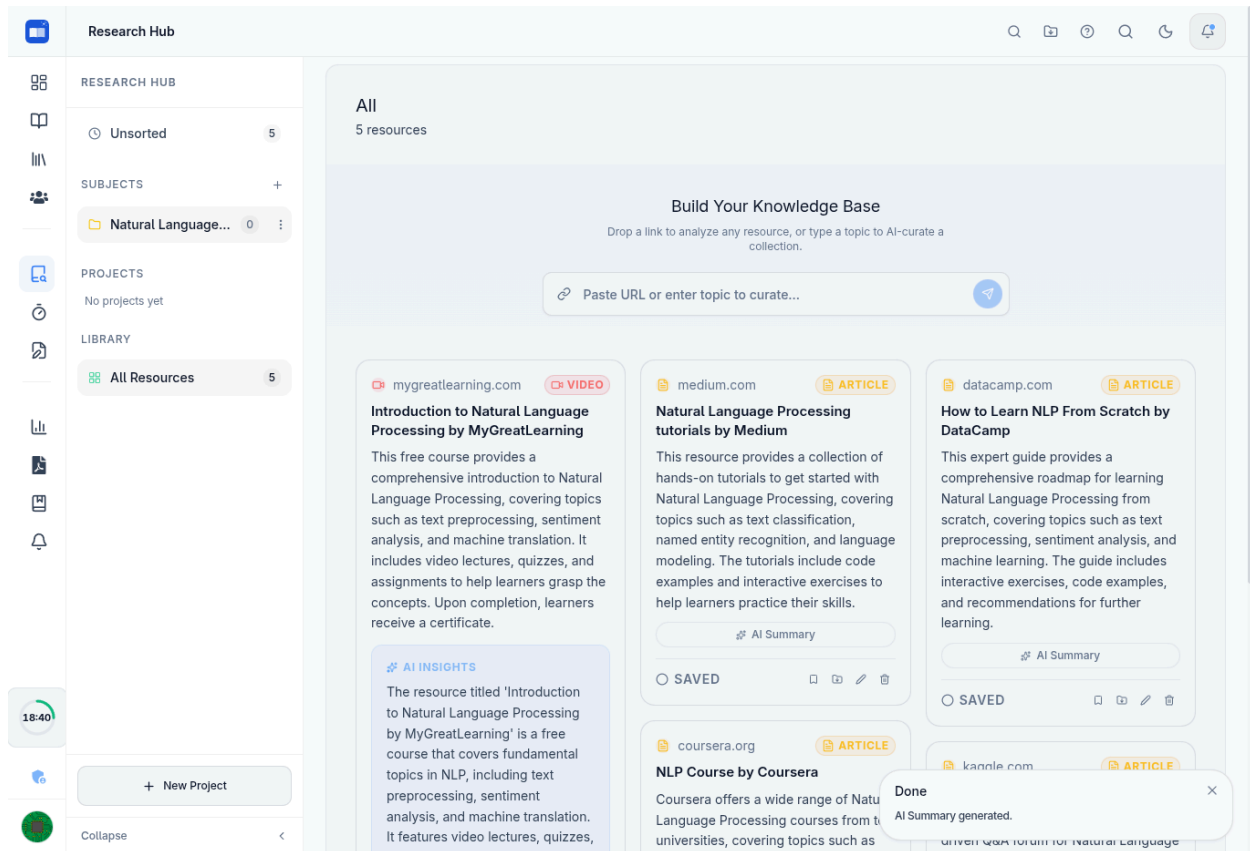


Figure 5.10: The Resource Library presenting a curated masonry grid of materials enhanced with AI-generated "Insights" for rapid relevance evaluation.

5.5.4 Study Plan

The Study Plan page organizes AI-generated tasks into a weekly/daily structure aligned to the exam date, with progress tracking and automatic surfacing of today's tasks.

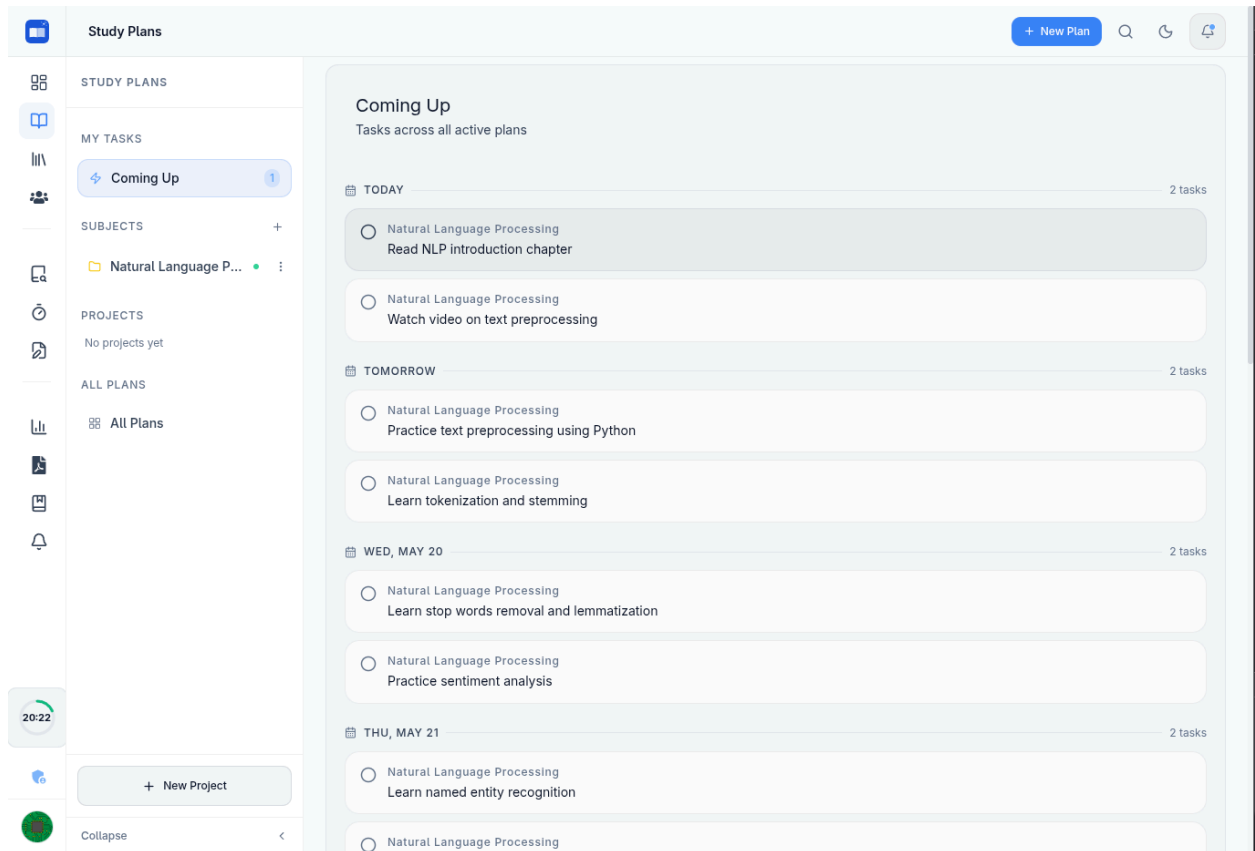


Figure 5.11: The Study Plan interface providing a timeline of study milestones, daily focus tasks, and step progress tracking.

5.5.5 Study Notes Library

The Neural Library serves as the long-term knowledge repository for captured insights.

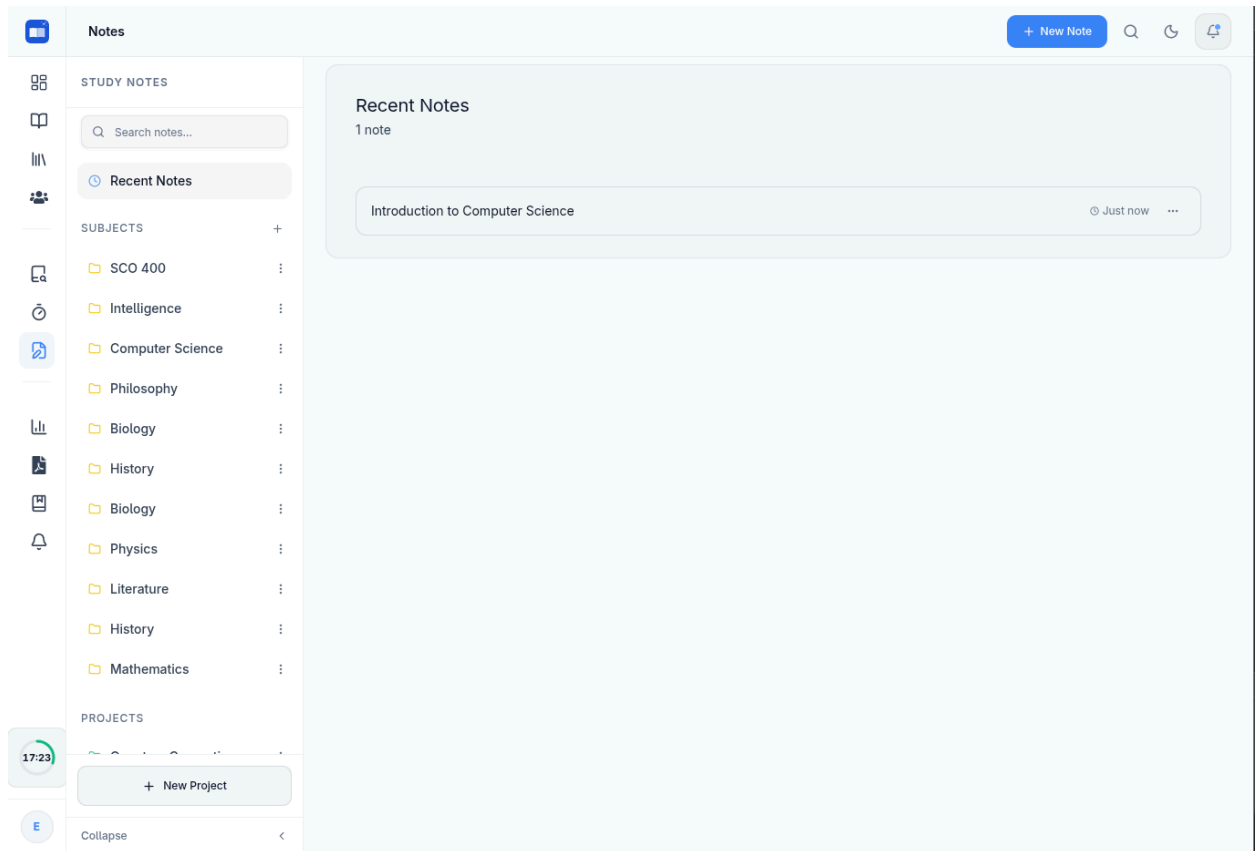


Figure 5.12: The Study Notes Library interface featuring card-based notes management with hierarchical folder structures and semantic tagging.

5.5.6 PDF Workshop

The PDF Workshop provides a split-pane workspace where users view uploaded PDFs alongside an AI chat assistant. It uses a RAG pipeline (local embeddings + vector similarity search + Groq LLM) to answer questions grounded in the document's content, with source page citations and the ability to save insights to notes.

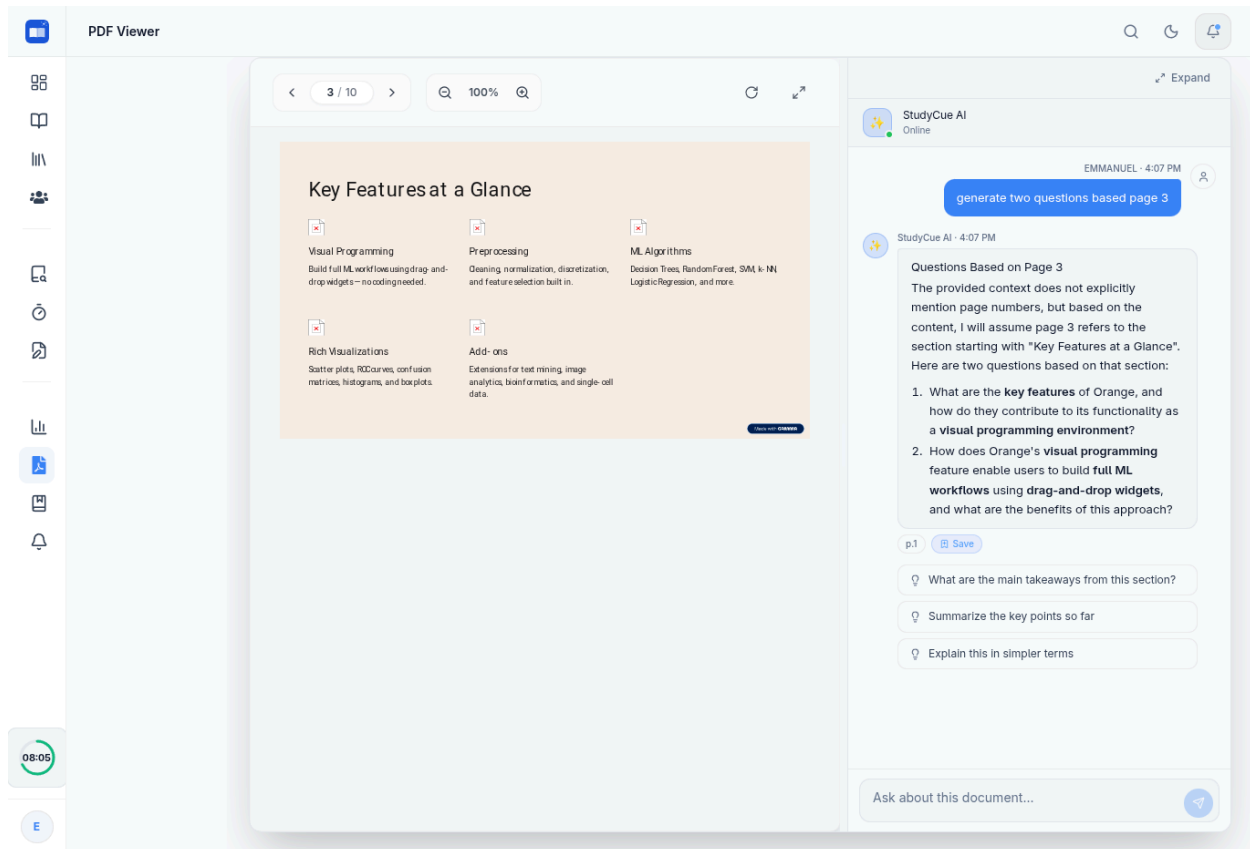


Figure 5.13: The PDF Workshop providing a split-pane workspace where users view uploaded PDFs alongside an AI chat assistant.

5.5.7 Community Q&A Interface

The Community module provides a dual-tab interface for peer knowledge sharing:

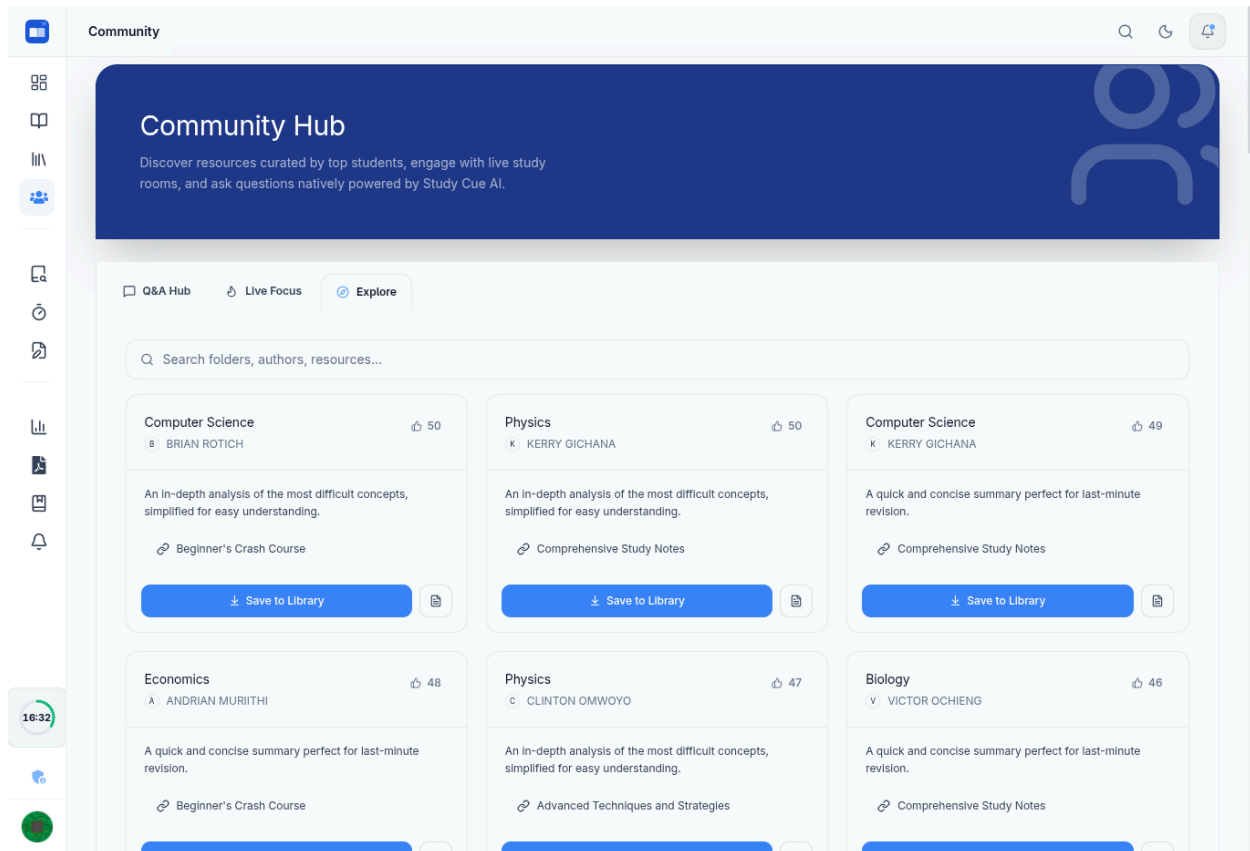


Figure 5.14: The Study Cue Community Hub interface featuring card-based resources engine with hierarchical folder structures and semantic tagging.

Tab 1 — Q&A Feed: Scrollable question cards showing avatar, title, tags, upvote/reply counts. Clicking expands the full thread with an inline reply composer and "Generate AI Answer" button. Filterable by tag or sortable by newest/most upvoted.

Tab 2 — Community Resources: Card-grid of shared resource folders showing topic, author, tags, upvote, and save counts. Students can clone folders into their Library via "Save to Library." Folders support nested subfolders.

Admin View: Reported content surfaces in the Admin Panel for review. Admins can soft-delete posts, removing them from the public feed without permanent deletion.

CHAPTER 6: SYSTEM IMPLEMENTATION

6.1 Introduction

This chapter describes the implementation of the Study Cue platform, covering the tools used for coding and testing, the testing approach, and the proposed change-over techniques. Development followed an Agile methodology with iterative sprints covering the Study Planner, Focus Timer, Research Hub, PDF Workshop, Notes Management, and Community Q&A module.

6.2 Tools Used for Coding and Testing

The platform was implemented using a modern JavaScript/TypeScript stack, as summarized in the table below:

Category	Tool	Purpose
IDE	Visual Studio Code / Cursor	Frontend and backend development
Version Control	Git and GitHub	Source code management and versioning
Frontend Framework	Next.js 14 & React 18	Server-side rendered, responsive UI
Styling & UI	Tailwind CSS, Radix UI, Framer Motion	Styling, accessible components, animations
Backend Framework	Node.js with Express.js	RESTful API endpoints
ORM	Mongoose	Object Data Modeling for MongoDB
Database	MongoDB	NoSQL persistent data storage

Authentication	NextAuth.js & JWT	User sessions, Google OAuth, credential logins
AI — LLM Inference	Groq API (Llama-3)	Natural language generation for Study Plan and Research Hub
AI — Embeddings	HuggingFace Inference API	Sentence embeddings for PDF semantic search (RAG)
AI — Resource Discovery	Tavily API	Academic-focused web search and resource curation
API Testing	Swagger & Postman	Endpoint documentation and testing
Analytics	PostHog	User behaviour tracking and funnel analysis
UI Testing	Chrome, Edge, Firefox	Cross-browser compatibility verification

All tools were used under open-source, free-tier, or student licenses, minimizing development costs while maintaining enterprise-grade architecture.

6.3 Testing Approach and Data Used

Testing was conducted continuously throughout the Agile development lifecycle using the following strategies:

6.3.1 Unit Testing

Backend utility functions and Mongoose model validations were tested, including user schema requirements, bcrypt password hashing, and study plan data structures. Frontend component logic was verified manually, focusing on the Focus Timer state management and Study Plan form validations.

6.3.2 Integration Testing

API endpoints were tested via Swagger and Postman to validate the full request/response lifecycle, covering user registration, study plan generation, PDF uploads, and Community Q&A posting. Mongoose queries were verified for correct relationship mapping between Users, Study Plans, and Community Resources. External integrations with Groq and Tavily APIs were tested for proper context generation and error fallback handling.

6.3.3 Functional Testing

Core user workflows were tested using synthetic data:

- **Study Planning Flow:** A student registered, entered a subject and exam date, and the system computed available study days, scanned existing Library content for context, and passed the assembled context to Groq (Llama-3), which generated a structured weekly JSON plan. The student then reviewed and completed daily tasks on the dashboard.
- **PDF RAG Flow:** A PDF textbook was uploaded, chunked, and embedded via HuggingFace. Queries returned Groq-powered contextual answers with source attribution, which were archived to the Notes module.
- **Resource Discovery Flow:** A topic query triggered the Tavily API to retrieve and AI-summarize curated academic resources, which were saved to the personal Library.
- **Admin Flow:** An admin logged in, reviewed and moderated Community Q&A posts, and monitored platform analytics via the PostHog dashboard.

Sample test data included synthetic student accounts, AI-generated study plans with modular tasks (e.g., "Calculus 101" covering Derivatives, Integrals, and Limits), academic PDFs chunked into 500-token segments, and seeded Community Q&A interactions.

6.3.4 User Acceptance Testing (UAT)

Volunteer university students participated in guided walkthroughs of the local deployment, evaluating ease of study plan creation, AI research clarity, and Focus Timer usability. Feedback on navigation transitions and dark mode contrast was iteratively incorporated into the final prototype.

6.3.5 Security Testing

Unauthenticated requests to protected API routes were verified to return HTTP 401/403 errors. Passwords were confirmed to be hashed using bcryptjs prior to database insertion. Server actions and Express routes were tested with malformed inputs to ensure proper error handling without exposing stack traces.

6.4 Proposed Change-over Techniques

6.4.1 Pilot Change-over (Recommended)

Study Cue was introduced to a controlled cohort of approximately 30 students for one academic semester. Students were onboarded via an interactive platform tour, used the system for a specific subject, and provided feedback monitored through engagement metrics. This low-risk approach allowed identification of performance bottlenecks and UX friction points before a wider rollout.

6.4.2 Parallel Change-over (Alternative)

Students used Study Cue alongside existing tools such as Notion or physical notebooks during a transitional period, gradually migrating their full workflow. While this minimized disruption to study routines, it risked duplicated effort and slower early adoption.

6.4.3 Training and Documentation

To support smooth onboarding, a "Getting Started" guide was pinned in the Community Q&A feed, contextual tooltips and an interactive onboarding sequence were implemented for first-time users, and "How-to" sections were documented on the platform's marketing pages.

CHAPTER 7: LIMITATIONS, CONCLUSIONS AND RECOMMENDATIONS

7.1 Limitations

The following limitations were identified during the development and initial testing phases of Study Cue:

1. **AI Dependency and Rate Limits:** The Research Hub and Study Plan generator rely heavily on third-party APIs (Google Gemini/Groq). Free-tier API rate limits could restrict simultaneous usage during high-traffic periods (e.g., finals week).
2. **File Storage Constraints:** The personal Library currently handles PDF uploads. Managing storage costs for large-scale document hosting across a massive user base was beyond the scope of this initial local prototype.
3. **Internet Dependency:** As a web-based platform with real-time AI and database queries, Study Cue requires a stable internet connection, limiting its utility for offline study sessions or in areas with poor connectivity.
4. **Time Constraints:** The project was developed within a strict timeframe, limiting the breadth of longitudinal UAT to measure actual GPA/academic performance improvements resulting from platform usage.
5. **Lack of Native Mobile App:** While the web interface is fully responsive, the lack of a native mobile application (iOS/Android) limits features like push notifications for study reminders and offline access.

7.2 Conclusions

The Study Cue project successfully engineered an all-in-one digital study companion. The system addressed key objectives identified during initial requirements gathering:

- Consolidated fragmented study tools into a single, cohesive dashboard featuring a custom Focus Timer, Study Planner, and document Library.
- Leveraged AI to reduce the cognitive load of organizing study materials, successfully generating structured, actionable study plans and providing instant, contextual academic research.
- Fostered collaborative learning through a Community Q&A and resource-sharing feed, promoting peer-to-peer academic support.

- Delivered a premium, highly responsive user interface prioritizing dark-mode aesthetics and distraction-free learning environments.

7.3 Recommendations

Based on project findings, the following recommendations are proposed for future iterations of Study Cue:

7.3.1 Technical Recommendations

1. **Implement Cloud Object Storage:** Integrate Amazon S3 or Cloudinary to sustainably handle student PDF and document uploads in the Library module.
2. **Develop a Native Mobile Application:** Utilize React Native or Flutter to build a companion app, enabling offline study tracking, persistent timer notifications, and on-the-go flashcard review.
3. **Advanced AI Features:** Implement Retrieval-Augmented Generation (RAG) so the AI can chat directly with a student's uploaded PDFs within their personal Library, providing highly personalized tutoring.

7.3.2 Educational and Content Recommendations

1. **Institutional Partnerships:** Collaborate with universities to integrate Study Cue with learning management systems (like Canvas or Moodle), allowing automatic import of syllabi into the Study Planner.
2. **Gamification:** Expand the dashboard progress rings into a comprehensive gamification system (e.g., academic streaks, badges, leaderboards) to incentivize consistent study habits.

7.3.3 Deployment Recommendations

1. **Public Cloud Deployment:** Transition from local development/Docker to a highly available public cloud infrastructure (e.g., Vercel for frontend, Render/AWS for backend) to support a public beta launch.
2. **Analytics Integration:** Fully implement PostHog or similar analytics to track user drop-off points within the study flow, allowing for data-driven UI improvements.

APPENDICES

APPENDIX A

QUESTIONNAIRE QUESTIONS FOR STUDENTS (USERS)

1. On a scale of 1–5, how overwhelmed do you feel when organizing your study schedule for a new semester?
2. Which tools do you currently use to manage your studying? (Select all that apply: Physical planner, Google Calendar, Notion, generic timers, none, other).
3. How often do you experience "study block" or struggle to break down a large topic into manageable learning steps?
4. How comfortable are you with using Artificial Intelligence (like ChatGPT) to assist in your academic research and planning?
5. Which features would you find most useful in an all-in-one study platform? (*Options: AI-generated study plans, built-in Pomodoro timer, peer-to-peer community Q&A, PDF/document storage, AI document summarization, progress tracking rings*).
6. Have you used any digital study platforms before? If yes, what did you like or dislike about them?
7. How likely are you to use a platform that integrates your study timer, notes, and research into one dashboard rather than switching between tabs?
8. What are your primary distractions when trying to study online?
9. In your own words, describe the biggest challenge you face when preparing for major exams.

APPENDIX B

QUESTIONNAIRE QUESTIONS FOR EDUCATORS / PEER TUTORS

1. What are the most common organizational struggles you observe in your students?
2. How frequently do students express difficulty in knowing *how* to study rather than *what* to study?
3. What barriers do students face when transitioning from attending lectures to independent study?
4. How effective do you think digital platforms are in improving student retention and consistency?
5. How concerned are you about the ethical use of AI by students in their academic workflow?
6. If an AI platform helped students break down your syllabus into daily study tasks, how do you think it would impact their performance?
7. What challenges do you face when trying to share supplemental study materials with your class?
8. How should a digital platform balance providing AI assistance with encouraging independent critical thinking? (*Options: Limiting direct answers, forcing active recall, focusing on planning rather than writing, promoting peer discussion*).

APPENDIX C

INTERESTING CODE SNIPPETS

1. Study Plan Generation — Hybrid Algorithm + AI Pipeline (Node.js/Groq)

```
// Step 1 — ALGORITHM: Calculate available study days

const daysUntilExam = Math.ceil(

  (new Date(examDate) - new Date()) / (1000 * 60 * 60 * 24)

);

// Step 2 — ALGORITHM: Build context from student's existing data

// Scans Library folders, Research Hub items, and uploaded PDFs

// Uses keyword regex to detect syllabi / course outlines

const folderContext = await buildFolderContext(userId, subject);

// e.g. returns: "SUBJECT OUTLINE: Week 1 - Kinematics, Week 2 - Dynamics...

//           Library resources: - Lecture Notes (PDF)..."

// Step 3 — ALGORITHM: Merge user-provided context with auto-gathered context

const combinedContext = [folderContext, userContext].filter(Boolean).join("\n\n");

// Step 4 — AI (Groq Llama-3): Generate personalised plan using pre-built context

const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });

const completion = await groq.chat.completions.create({

  model: 'llama-3.3-70b-versatile',

  messages: [

    { role: 'system', content: 'You are an expert study planner. Respond in JSON.' },

    { role: 'user', content: `Create a ${daysUntilExam}-day plan for ${subject}.\n${combinedContext}` },

  ],

  response_format: { type: 'json_object' },

  temperature: 0.7,
```

```

});

// Step 5 — ALGORITHM: Validate AI output structure before persisting
const parsedPlan = JSON.parse(completion.choices[0].message.content);
if (!parsedPlan.overview || !parsedPlan.weeklyPlans) {
  throw new Error('Invalid plan structure returned by AI');
}

// Step 6 — ALGORITHM: Auto-link a Library folder to the new study plan
const plan = new StudyPlan({ userId, ...parsedPlan, isActive: true, progress: 0 });
await plan.save();

```

2. PDF RAG — Document Embedding & Semantic Search (HuggingFace)

```

import { HfInference } from '@huggingface/inference';

const hf = new HfInference(process.env.HUGGINGFACE_API_KEY);

// Generate embedding vector for a text chunk

export async function embedText(text) {
  const result = await hf.featureExtraction({
    model: 'sentence-transformers/all-MiniLM-L6-v2',
    inputs: text,
  });

  return result; // returns a float[] embedding vector
}

// Cosine similarity to find the most relevant chunks for a user query

export function cosineSimilarity(vecA, vecB) {
  const dot = vecA.reduce((sum, a, i) => sum + a * vecB[i], 0);
  const magA = Math.sqrt(vecA.reduce((sum, a) => sum + a * a, 0));

```

```

const magB = Math.sqrt(vecB.reduce((sum, b) => sum + b * b, 0));

return dot / (magA * magB);

}

```

3. Focus Timer State Management (React/Zustand)

```

import { create } from 'zustand';

interface TimerState {

  timeLeft: number;

  isRunning: boolean;

  startTimer: () => void;

  pauseTimer: () => void;

  tick: () => void;

}

export const useTimerStore = create<TimerState>((set) => ({

  timeLeft: 25 * 60, // 25 minutes default (Pomodoro)

  isRunning: false,

  startTimer: () => set({ isRunning: true }),

  pauseTimer: () => set({ isRunning: false }),

  tick: () => set((state) => ({

    timeLeft: state.timeLeft > 0 ? state.timeLeft - 1 : 0

  })),

}));

```

APPENDIX D

TECHNICAL GUIDE AND USER MANUAL

1. System Requirements

- **Operating System:** Windows 10+, macOS 11+, or modern Linux distributions.
- **Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge (latest versions recommended).
- **Network:** Stable broadband internet connection required for AI querying and database sync.

2. User Registration and Login

- Navigate to www.studycue.me (or localhost:3000 for local deployment).
- Click "**Sign Up**" to create a new account using an email and password, or click "**Continue with Google**" for Single Sign-On (SSO).
- Upon successful login, you will be redirected to the main Dashboard.

3. Generating a Study Plan

- On the Dashboard, click "**New Plan**".
- Enter the Subject name (e.g., "Organic Chemistry") and select your target Exam Date.
- Click "**Generate**". The AI will process the request and populate your dashboard with a structured timeline of study tasks.
- Click the checkbox next to a task to mark it as completed and update your progress ring.

4. Using the Focus Timer

- Navigate to the **Timer** page via the left sidebar.
- Select your preferred mode (e.g., Pomodoro 25/5).
- Click "**Start**" to begin the countdown. You can pause the timer at any point.

5. Accessing the Research Hub

- Navigate to the **Research Hub**.
- Type your academic query into the search bar.

- The AI will generate a comprehensive summary and provide relevant academic context.
You can click "Save" to add this research to your personal Library.

REFERENCES

1. Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. <https://agilemanifesto.org/>
2. Bozkurt, A., & Sharma, R. C. (2020). Emergency remote teaching in a time of global crisis due to CoronaVirus pandemic. *Asian Journal of Distance Education*, 15(1), i–vi. <https://www.asianjde.org/ojs/index.php/AsianJDE/article/view/447>
3. Braun, V., & Clarke, V. (2013). *Successful Qualitative Research: A Practical Guide for Beginners*. SAGE Publications.
4. Chen, Y., & Lin, H. (2023). Integrating large language models into self-regulated learning platforms. *International Journal of Artificial Intelligence in Education*, 12(1), 45–67.
5. Creswell, J. W., & Creswell, J. D. (2018). *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches* (5th ed.). SAGE Publications.
6. Crockford, D. (2008). *JavaScript: The Good Parts*. O'Reilly Media.
7. Floridi, L., & Cowls, J. (2019). A unified framework of five principles for AI ethics. *Harvard Data Science Review*, 1(1). <https://doi.org/10.1162/99608f92.8cd5501d>
8. Google for Developers. (2024). *Gemini API documentation*. Retrieved May 18, 2026, from <https://ai.google.dev/docs>
9. Groq. (2024). *Groq LPU inference engine: Fast AI inference*.
10. Kariuki, M., & Wanjiru, E. (2022). Barriers to digital tool adoption among university students in Kenya. *African Journal of Information Systems*, 14(2), 89–104.
11. Kasneci, E., Seßler, K., Küchemann, S., & Kasneci, G. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274. <https://doi.org/10.1016/j.lindif.2023.102274>
12. Lucide. (2024). *Beautiful & consistent icons*. Retrieved May 18, 2026, from <https://lucide.dev/>
13. Luckin, R., Holmes, W., Griffiths, M., & Forcier, L. B. (2016). *Intelligence unleashed: An argument for AI in education*. Pearson. <https://discovery.ucl.ac.uk/id/eprint/1475116/>
14. MongoDB. (2024). *MongoDB: The most popular database for modern apps*. Retrieved May 18, 2026, from <https://mongoosejs.com/>